

Sistema de Recomendação de Filmes com PCA

Recomendações Híbridas Utilizando Padrões de Avaliação e Conteúdo

Guilherme Desiderio dos Santos
Instituto de Computação, Centro de Ciências da Matemática e da Natureza
Universidade Federal do Rio de Janeiro

Rio de Janeiro – RJ, julho de 2026

Arquivo analisado: Recomendador de Filmes.ipynb

Sumário

1	Motivação	3
2	Ferramentas Utilizadas de Análise de Dados e Aprendizado de Máquina	4
2.1	Matriz usuário-filme	4
2.2	Normalização por usuário	4
2.3	PCA e TruncatedSVD	5
2.4	Similaridade de cosseno	5
2.5	TF-IDF para gêneros	5
2.6	Precision@K	6
3	Modelo Híbrido e Construção do Sistema de Recomendação	6
3.1	Representação esparsa dos dados	6
3.2	Normalização das avaliações	6
3.3	Redução de dimensionalidade via TruncatedSVD	7
3.4	Similaridade no espaço latente	7
3.5	Similaridade por conteúdo com TF-IDF	9
3.6	Score híbrido e função de recomendação	9
3.7	Considerações sobre o método	10

4	Experimento	11
4.1	Base de dados e exploração inicial	11
4.2	Limpeza e divisão treino-teste	12
4.3	Resultados do PCA e das visualizações	13
4.4	Resultados quantitativos de recomendação	14
4.5	Análise qualitativa das recomendações	16
4.6	Limitações e informações ausentes	16
5	Conclusão	17
	Bibliografia	18

1 Motivação

Sistemas de recomendação são ferramentas centrais em plataformas digitais que precisam organizar catálogos grandes e apresentar ao usuário itens relevantes. Em um catálogo com dezenas de milhares de filmes, como o MovieLens ml-32M, consultar manualmente todos os títulos é inviável. O objetivo do recomendador desenvolvido em `Recomendador de Filmes.ipynb` é reduzir essa complexidade: a partir de um filme de referência, o sistema retorna outros filmes considerados semelhantes. O problema pode ser formulado a partir de uma tabela de avaliações. Cada linha registra que um usuário atribuiu uma nota a um filme. Se essa tabela for transformada em uma matriz, cada linha representa um usuário, cada coluna representa um filme e cada célula contém a nota dada. Essa representação é natural, mas traz um obstáculo imediato: a matriz é extremamente grande e quase completamente vazia. No projeto analisado, após a limpeza, a matriz possui 200.948 usuários e 31.961 filmes, resultando em mais de 6,4 bilhões de possíveis posições. Destas, apenas 31.842.705 possuem uma avaliação real. A sparsidade calculada no notebook é de 99,5042%. Essa característica torna inadequado trabalhar com uma matriz densa. Guardar explicitamente todas as posições vazias desperdiçaria memória e deixaria operações de similaridade muito custosas. O notebook, portanto, trata o problema como um problema de escala: como aprender relações úteis entre filmes sem materializar todos os pares de usuários e filmes? Uma solução clássica é projetar os filmes em um espaço latente de menor dimensão. Em vez de comparar filmes diretamente por milhões de avaliações esparsas, o modelo aprende componentes que resumem padrões de coavaliação. Filmes avaliados de maneira parecida por grupos semelhantes de usuários tendem a ficar próximos nesse espaço. No projeto, essa redução é feita com `TruncatedSVD`, usado como alternativa operacional ao PCA tradicional, pois trabalha diretamente sobre matrizes esparsas. Entretanto, usar apenas avaliações pode deixar o sistema vulnerável a ruído, baixa cobertura e dependência excessiva do histórico coletivo. Por isso, o projeto combina duas fontes de informação:

- **Filtragem colaborativa via PCA/TruncatedSVD:** aprende proximidade entre filmes a partir dos padrões de avaliação dos usuários.
- **Filtragem por conteúdo via TF-IDF:** representa os gêneros dos filmes como vetores e mede proximidade textual entre eles.

A motivação do trabalho é construir um recomendador híbrido que aproveite as duas perspectivas. A primeira tenta capturar o comportamento real dos usuários; a segunda introduz uma noção explícita de conteúdo. A combinação final é controlada por um parâmetro α , que define o peso do componente colaborativo em relação ao componente de conteúdo. Em resumo, o projeto busca responder três perguntas principais:

1. Como representar uma base massiva e esparsa de avaliações sem inviabilizar o processamento?
2. Como transformar avaliações e gêneros em medidas comparáveis de similaridade entre filmes?
3. Como avaliar se as recomendações geradas realmente antecipam filmes que os usuários gostariam?

2 Ferramentas Utilizadas de Análise de Dados e Aprendizado de Máquina

Esta seção apresenta os conceitos utilizados no notebook antes da descrição do modelo. A organização segue a mesma lógica do desenvolvimento do projeto: primeiro a representação dos dados, depois a redução de dimensionalidade, a similaridade, a abordagem de conteúdo e, por fim, a métrica de avaliação.

2.1 Matriz usuário-filme

Considere um conjunto de usuários U e um conjunto de filmes I . A base de avaliações pode ser representada por uma matriz R , em que:

$R[u, i]$ = nota atribuída pelo usuário u ao filme i

Quando um usuário não avaliou um filme, a entrada correspondente fica vazia. Em bases reais, como o MovieLens ml-32M, isso ocorre na maior parte das posições. Por isso, o notebook usa `csr_matrix`, estrutura esparsa que armazena apenas valores existentes. No projeto, a matriz construída após a limpeza possui:

Tabela 1: Dimensões e sparsidade da matriz usuário-filme após a limpeza

Grandeza	Valor
Usuários	200.948
Filmes no modelo	31.961
Avaliações armazenadas	31.842.705
Posições possíveis	6.422.499.028
Sparsidade	99,5042%

Essa decisão é fundamental: sem uma estrutura esparsa, o modelo exigiria memória incompatível com um ambiente comum de execução.

2.2 Normalização por usuário

Usuários diferentes usam a escala de notas de maneiras diferentes. Um usuário pode ser mais generoso e dar notas altas com frequência; outro pode reservar notas altas para poucos filmes. Comparar notas brutas sem considerar esse comportamento introduz viés. O notebook corrige esse efeito subtraindo, de cada avaliação, a média do próprio usuário:

$$r'_{ui} = r_{ui} - \text{media}(u)$$

Assim, uma nota passa a indicar preferência relativa. Um valor positivo significa que o usuário avaliou aquele filme acima de sua média pessoal. Um valor negativo indica que avaliou abaixo de

sua média. Essa centralização também aproxima a preparação dos dados da hipótese exigida por métodos de decomposição como PCA.

2.3 PCA e TruncatedSVD

O PCA busca representar dados de alta dimensão em um número menor de componentes, preservando parte da variabilidade dos dados. Em termos intuitivos, ele encontra direções principais que resumem padrões recorrentes. O PCA tradicional, porém, costuma exigir uma matriz densa. Como a matriz do projeto é grande e esparsa, o notebook usa **TruncatedSVD**, que consegue operar diretamente nesse formato. Como as avaliações foram centralizadas por usuário, o uso do **TruncatedSVD** cumpre o papel prático de obter um espaço latente semelhante ao produzido por PCA. O modelo é ajustado inicialmente com 100 componentes para estudar a variância explicada. Em seguida, o projeto escolhe 50 componentes como dimensão final do espaço latente:

Tabela 2: Variância explicada acumulada por número de componentes

Componentes	Variância explicada acumulada
10	9,1%
20	12,0%
50	17,4%
100	23,2%

A variância acumulada é relativamente baixa, mas isso é esperado em dados de recomendação muito esparsos. Parte significativa da variação se deve a preferências individuais, ruído e baixa sobreposição entre usuários. Mesmo assim, os componentes iniciais podem capturar sinal suficiente para organizar filmes em vizinhanças úteis.

2.4 Similaridade de cosseno

Após a decomposição, cada filme passa a ser representado por um vetor de 50 dimensões. Para comparar dois filmes, o notebook utiliza similaridade de cosseno:

$$\text{sim}(i, j) = \cos(\text{theta}) = (\mathbf{v}_i \cdot \mathbf{v}_j) / (||\mathbf{v}_i|| \ ||\mathbf{v}_j||)$$

O valor aumenta quando os vetores apontam para direções semelhantes. Essa métrica é adequada porque o interesse principal é a orientação dos vetores latentes, não necessariamente sua magnitude absoluta. Calcular a similaridade entre todos os pares de filmes exigiria uma matriz de aproximadamente 31.961 x 31.961 posições, ou seja, mais de 1 bilhão de pares. O notebook evita esse custo usando **NearestNeighbors**, que calcula vizinhos sob demanda.

2.5 TF-IDF para gêneros

A filtragem por conteúdo representa cada filme a partir de seus gêneros. No MovieLens, os gêneros aparecem em uma única string separada por |, por exemplo:

Action|Adventure|Sci-Fi

O notebook usa `TfidfVectorizer` com tokenizador personalizado para separar exatamente por |. Isso evita quebrar termos como `Sci-Fi` em partes indevidas. Foram encontrados 20 termos de gênero no universo modelado, incluindo `action`, `adventure`, `comedy`, `drama`, `sci-fi`, `thriller`, `war`, `western` e a categoria (`no genres listed`).

2.6 Precision@K

Para avaliar o sistema, o notebook usa `Precision@K`. A ideia é verificar, entre os K filmes recomendados, quantos aparecem no conjunto de filmes que o usuário efetivamente avaliou bem no teste.

$\text{Precision@K} = \text{recomendações relevantes no top-K} / K$

No projeto, uma avaliação é considerada positiva quando a nota é maior ou igual a 4,0. Para cada usuário avaliado, o sistema usa alguns filmes bem avaliados no treino como sementes, gera recomendações e compara o resultado com os filmes bem avaliados pelo mesmo usuário no teste.

3 Modelo Híbrido e Construção do Sistema de Recomendação

O modelo desenvolvido no notebook é híbrido porque combina uma visão colaborativa, derivada das avaliações, com uma visão de conteúdo, derivada dos gêneros. A construção é sequencial: cada etapa produz artefatos usados pela etapa seguinte.

3.1 Representação esparsa dos dados

O notebook começa lendo `ratings.csv` e `movies.csv`. A tabela `ratings.csv` contém `userId`, `movieId`, `rating` e `timestamp`. A tabela `movies.csv` contém `movieId`, `title` e `genres`. Depois da leitura, os identificadores de usuários e filmes são convertidos para categorias. Essa conversão permite transformar IDs possivelmente não sequenciais em códigos inteiros consecutivos, adequados para montar a matriz esparsa. O resultado é uma matriz X em que:

linhas = usuários
colunas = filmes
valores = avaliações observadas

Essa matriz é a base da filtragem colaborativa. Ela preserva apenas avaliações existentes e deixa implícita a ausência de avaliação.

3.2 Normalização das avaliações

Em seguida, o notebook calcula a média de notas de cada usuário e subtrai essa média de todas as avaliações do mesmo usuário. A matriz normalizada `X_norm` passa a armazenar desvios em

relação ao comportamento médio individual. Essa etapa é importante por dois motivos. Primeiro, reduz o viés causado por usuários mais generosos ou mais rigorosos. Segundo, torna os dados mais adequados à decomposição latente, pois o método passa a trabalhar com preferências relativas.

3.3 Redução de dimensionalidade via TruncatedSVD

Com a matriz normalizada, o notebook ajusta um `TruncatedSVD` com `MAX_K = 100`. Esse primeiro ajuste serve para observar como a variância acumulada cresce à medida que mais componentes são adicionados.

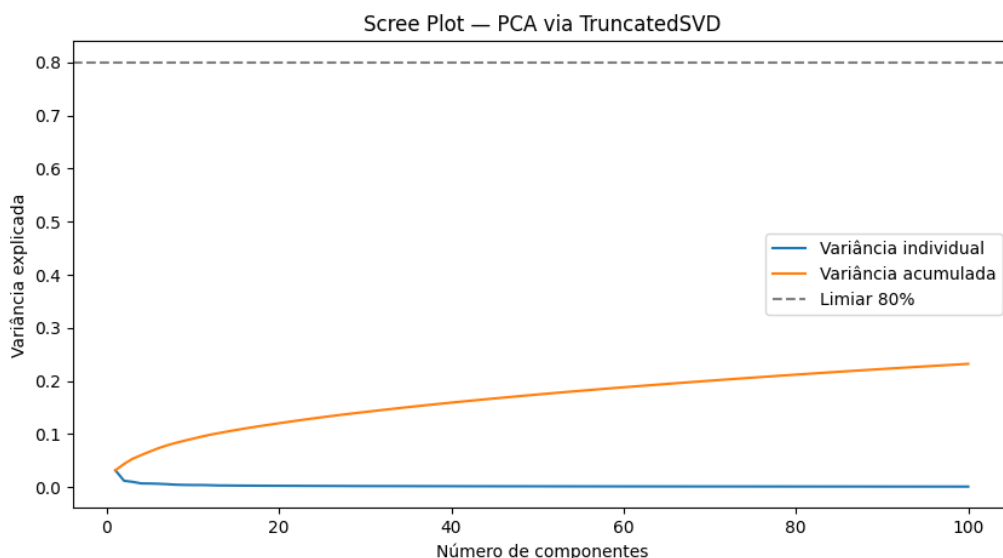


Figura 1: Scree plot com variância individual e acumulada.

O gráfico mostra que o ganho marginal diminui conforme o número de componentes aumenta. O projeto adota `N_COMPONENTS = 50`, produzindo:

Tabela 3: Dimensões das matrizes de fatores latentes

Matriz	Dimensão
<code>user_factors</code>	200.948 x 50
<code>movie_loadings</code>	31.961 x 50

Os vetores de `movie_loadings` são a representação latente dos filmes. É sobre eles que a similaridade colaborativa passa a ser calculada.

3.4 Similaridade no espaço latente

Com os filmes representados em 50 dimensões, o projeto ajusta um índice `NearestNeighbors` com métrica de cosseno. Esse índice substitui a matriz completa de similaridade. Para fins de inspeção

visual, o notebook calcula a similaridade apenas entre os 20 filmes mais avaliados e constrói um heatmap. A escolha de uma amostra pequena torna a visualização legível e evita o custo de uma comparação total.

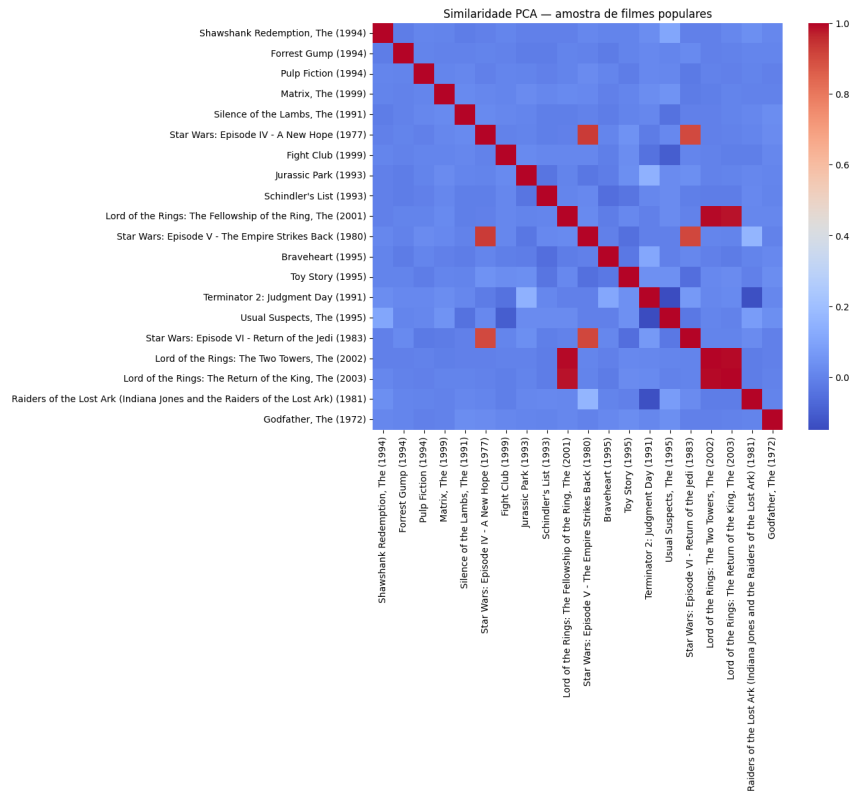


Figura 2: Heatmap de similaridade PCA entre filmes populares.

O mesmo espaço latente também é visualizado por meio de uma projeção dos dois primeiros componentes para uma amostra de 3.000 filmes, coloridos pelo gênero principal.

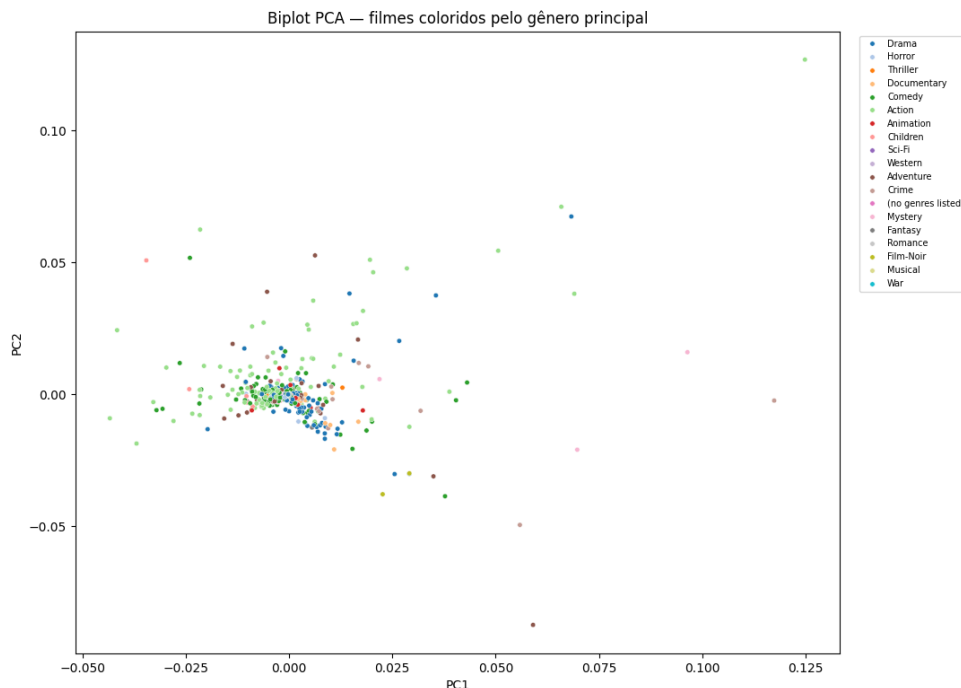


Figura 3: Biplot PCA de filmes coloridos por gênero principal.

Essas visualizações não substituem a avaliação quantitativa, mas ajudam a verificar se a decomposição produziu uma organização minimamente interpretável.

3.5 Similaridade por conteúdo com TF-IDF

A segunda visão do modelo usa os gêneros dos filmes. Cada string de gêneros é transformada em um vetor TF-IDF. Em seguida, outro índice **NearestNeighbors** é treinado sobre essa matriz de conteúdo. Essa etapa tem uma função complementar. A representação por conteúdo não depende de coavaliações, mas também é mais limitada, porque os gêneros são descritores amplos. Dois filmes de drama podem ser muito diferentes, e dois filmes de gêneros distintos podem ter públicos semelhantes. Por isso, o conteúdo entra como parte do score, não como única fonte de recomendação.

3.6 Score híbrido e função de recomendação

A função central do sistema calcula candidatos em duas etapas. Primeiro, busca vizinhos próximos segundo o PCA e vizinhos próximos segundo o conteúdo. Depois, une esses candidatos e calcula um score híbrido:

$$\text{score_hibrido}(i, j) = \alpha \cdot \text{sim_pca}(i, j) + \beta \cdot \text{sim_conteudo}(i, j)$$

com:

$$\beta = 1 - \alpha$$

No notebook, o valor padrão é $\alpha = 0,6$ e $\beta = 0,4$. Isso significa que, inicialmente, o projeto dá mais peso ao padrão colaborativo do que ao conteúdo, mas ainda preserva influência dos gêneros. A função `recomendar(titulo, alpha=0.6, beta=0.4, top_n=10, m_candidatos=50)` recebe o título exato de um filme e retorna uma tabela com:

Tabela 4: Colunas retornadas pela função de recomendação

Coluna	Significado
<code>title</code>	Filme recomendado
<code>sim_pca</code>	Similaridade no espaço latente colaborativo
<code>sim_content</code>	Similaridade pelos gêneros
<code>sim_hibrida</code>	Combinação ponderada usada na ordenação

O uso de candidatos evita comparar o filme de entrada com todos os títulos do catálogo. Isso mantém a função viável mesmo com mais de 30 mil filmes modelados.

É importante destacar que, a partir da divisão treino-teste (Seção 4.2), o sistema passa a operar sobre dois modelos distintos, ajustados de forma independente. O primeiro, construído nas Seções 3.1–3.4 sobre o conjunto de dados limpo completo (31.842.705 avaliações), foi utilizado apenas para a exploração inicial do espaço latente: o heatmap de similaridade (Figura 2) e o biplot por gênero (Figura 3). O segundo modelo é reajustado exclusivamente sobre o conjunto de treino (25.474.164 avaliações, 80% do total), com seu próprio `TruncatedSVD`, índices de vizinhos e vetorização TF-IDF, independentes dos primeiros.

É esse segundo modelo e não o primeiro que alimenta a função `recomendar()` e, portanto, sustenta todos os resultados apresentados nas Seções 4.4 e 4.5, incluindo a avaliação quantitativa por `Precision@K` e os exemplos qualitativos de recomendação. Essa distinção evita ambiguidade sobre qual conjunto de dados efetivamente fundamenta os resultados finais do sistema.

3.7 Considerações sobre o método

O método é eficiente por combinar três decisões técnicas coerentes:

- A matriz de avaliações é mantida em formato esparsa.
- A redução de dimensionalidade é feita com `TruncatedSVD`, evitando densificação.
- A busca de vizinhos é feita sob demanda, sem materializar a matriz completa de similaridade.

Essas escolhas preservam o espírito do PCA, mas adaptam a implementação às restrições práticas do dataset. O resultado é um pipeline capaz de processar dezenas de milhões de avaliações em um ambiente comum, desde que haja memória suficiente para as tabelas principais. O método também possui limites. O uso de gêneros como conteúdo é simples e interpretável, mas pouco granular. Além disso, a avaliação descarta usuários e filmes que aparecem apenas no teste, o que significa que o projeto não resolve o problema de cold start. Ainda assim, para o objetivo de construir e validar um recomendador híbrido sobre o MovieLens, a arquitetura é consistente.

4 Experimento

4.1 Base de dados e exploração inicial

O experimento utiliza o MovieLens ml-32M. A leitura inicial confirmou:

Tabela 5: Dimensões dos arquivos do MovieLens ml-32M

Arquivo	Registros ou itens
<code>ratings.csv</code>	32.000.204 avaliações
Usuários distintos	200.948
Filmes avaliados distintos	84.432
<code>movies.csv</code>	87.585 filmes

A distribuição das notas mostra concentração entre 3,0 e 4,0, com quantidade expressiva de avaliações 4,0 e 5,0.

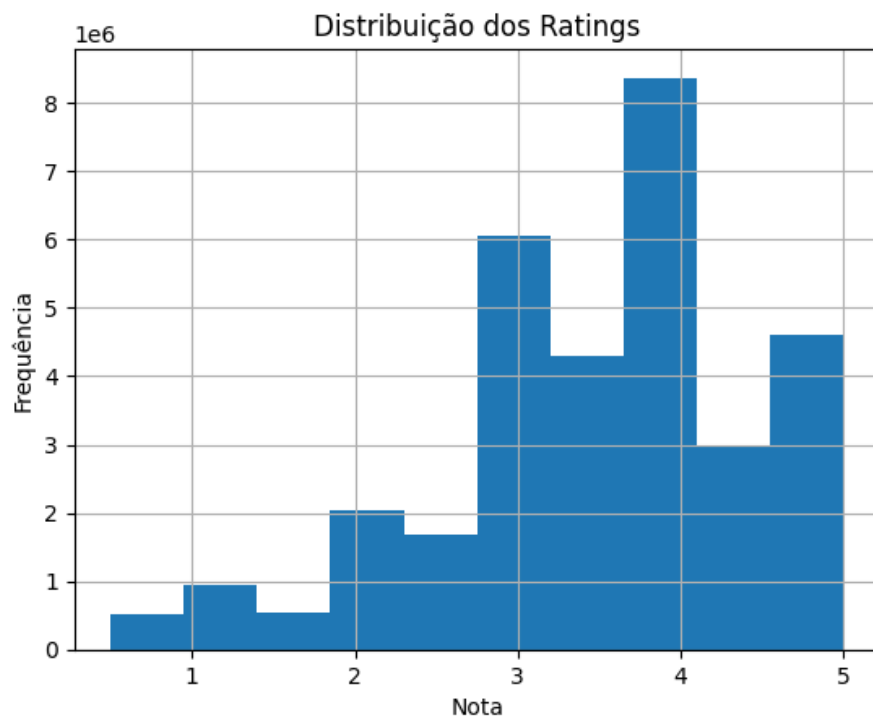


Figura 4: Distribuição das notas no conjunto de avaliações.

A contagem de notas no arquivo bruto foi:

Tabela 6: Distribuição das avaliações por nota no arquivo bruto

Nota	Quantidade
0,5	525.132
1,0	946.675
1,5	531.063
2,0	2.028.622
2,5	1.685.386
3,0	6.054.990
3,5	4.290.105
4,0	8.367.654
4,5	2.974.000
5,0	4.596.577

Os filmes mais avaliados incluem *The Shawshank Redemption* (1994), *Forrest Gump* (1994), *Pulp Fiction* (1994), *The Matrix* (1999) e *The Silence of the Lambs* (1991). Esse padrão é esperado em um dataset de grande escala: uma parcela pequena do catálogo concentra muitas avaliações, enquanto uma quantidade grande de filmes possui pouco suporte estatístico.

Para facilitar a reprodutibilidade, os principais hiperparâmetros e constantes usados no notebook são resumidos a seguir:

Tabela 7: Principais hiperparâmetros e constantes do experimento

Hiperparâmetro	Valor	Onde é usado
<code>N_COMPONENTS</code>	50	Dimensão final do espaço latente
<code>MAX_K</code>	100	Ajuste inicial do SVD para o scree plot
α (padrão / ótimo)	0,6 / 0,9	Peso do componente colaborativo
β	$1 - \alpha$	Peso do componente de conteúdo
<code>m_candidatos</code>	50	Vizinhos buscados em cada visão
<code>N_SEMENTES</code>	5	Filmes positivos usados como semente
<code>LIMIAR_POSITIVO</code>	4,0	Nota mínima considerada positiva
<code>N_USUARIOS_AVALIACAO</code>	1.000	Amostra para Precision@K
Amostra p/ sensibilidade de α	200 usuários	Varreduras de α
<code>test_size</code>	0,2 (80/20)	Proporção treino-teste
<code>random_state</code>	42	Reprodutibilidade das etapas estocásticas
Filtro mínimo por filme	10 avaliações	Limpeza inicial da base

4.2 Limpeza e divisão treino-teste

O notebook remove filmes com menos de 10 avaliações. A justificativa é que filmes com poucas interações têm similaridade pouco confiável e tendem a introduzir ruído.

Tabela 8: Impacto da limpeza sobre avaliações e filmes

Etapa	Avaliações	Filmes
Antes da limpeza	32.000.204	84.432 filmes avaliados
Depois da limpeza	31.842.705	31.961 filmes
Removido	157.499 avaliações	52.471 filmes

Embora a limpeza remova apenas cerca de 0,49% das avaliações, ela reduz fortemente o número de filmes. Isso indica que muitos títulos possuem pouquíssimas avaliações. Para avaliar o modelo de maneira honesta, o notebook divide os dados limpos em treino e teste:

Tabela 9: Distribuição das avaliações entre treino e teste

Conjunto	Avaliações
Treino	25.474.164
Teste	6.368.541

Uma ressalva importante sobre essa divisão é que ela é feita de forma aleatória, avaliação por avaliação, e não de forma temporal. Isso significa que um mesmo usuário pode ter avaliações distribuídas tanto no treino quanto no teste, e um mesmo filme pode aparecer nos dois conjuntos o que difere de um cenário real de uso, no qual o sistema precisaria recomendar filmes a partir apenas do histórico passado do usuário, sem qualquer acesso a avaliações futuras.

A divisão aleatória é adequada ao objetivo deste trabalho, que é avaliar a qualidade geral do ranking produzido pelo modelo híbrido, mas não simula a passagem do tempo. Um protocolo de avaliação temporal, por exemplo ordenando as avaliações por `timestamp` e reservando as mais recentes de cada usuário como teste, representaria melhor o cenário de uso real e constitui uma extensão natural deste trabalho. A divisão é feita antes da reconstrução do modelo final. Depois disso, o PCA, o TF-IDF e os índices de vizinhos são refeitos usando apenas o conjunto de treino. Essa decisão evita vazamento de dados: o modelo avaliado não deve ter aprendido a partir das mesmas avaliações usadas para medir sua qualidade.

4.3 Resultados do PCA e das visualizações

O scree plot indica que 50 componentes explicam 17,4% da variância acumulada e 100 componentes explicam 23,2%. Em um primeiro contato, esses valores podem parecer baixos. Porém, em recomendação, a matriz costuma ser muito esparsa e ruidosa. O interesse do modelo não é reconstruir perfeitamente todas as notas, mas capturar vizinhanças úteis entre filmes. O heatmap de filmes populares permite observar blocos de maior similaridade, embora a interpretação dependa dos filmes selecionados. Já o biplot ajuda a verificar que os dois primeiros componentes não se reduzem simplesmente aos gêneros declarados. Isso é coerente com a proposta colaborativa: o espaço latente deve refletir padrões de avaliação, que podem coincidir parcialmente com gênero, popularidade, época, franquia, público-alvo e outros fatores não explicitados nos metadados.

4.4 Resultados quantitativos de recomendação

A avaliação usa 1.000 usuários sorteados. Destes, 978 puderam ser avaliados efetivamente, pois precisavam ter filmes positivos no treino e no teste. Uma avaliação positiva é definida como nota maior ou igual a 4,0. O resultado médio obtido foi:

Tabela 10: Resultados médios de Precision@K

Métrica	Valor
Precision@5	0,045399
Precision@10	0,039059

Em termos percentuais, isso corresponde a aproximadamente 4,54% em P@5 e 3,91% em P@10. Os valores são baixos em termos absolutos, mas devem ser interpretados considerando a dificuldade do problema: catálogo com 31.961 filmes, matriz 99,5% esparsa, poucas sementes por usuário e teste feito sobre interações ocultas.

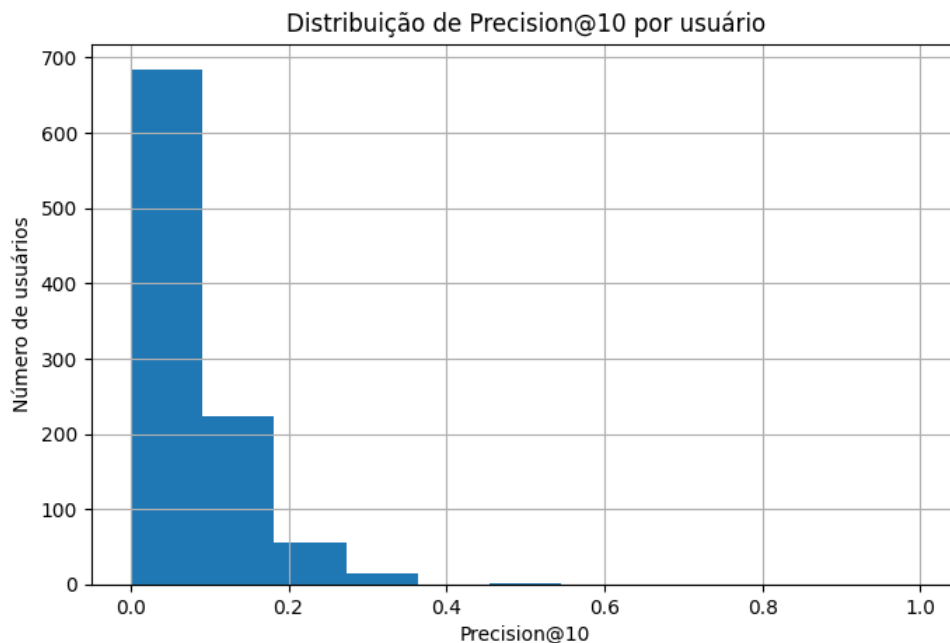


Figura 5: Distribuição da Precision@10 entre usuários avaliados.

O notebook também realiza uma análise de sensibilidade do parâmetro α . Para isso, testa valores de 0,0 a 1,0 em passos de 0,1, usando uma subamostra de 200 usuários.

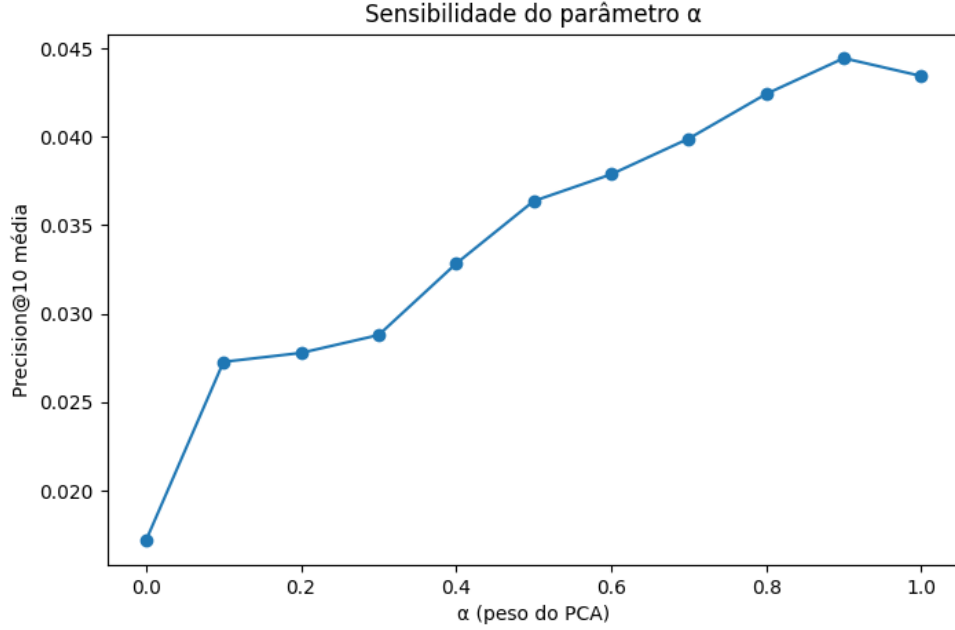


Figura 6: Precision@10 média em função do parâmetro alfa.

Os principais valores observados foram:

Tabela 11: Sensibilidade da Precision@10 ao parâmetro α

α	Precision@10 média
0,0	0,017172
0,3	0,028788
0,6	0,037879
0,9	0,044444
1,0	0,043434

Vale notar que os extremos da tabela de sensibilidade acima já funcionam como baselines naturais de comparação para o modelo híbrido. O valor $\alpha = 0,0$ corresponde a um recomendador puramente baseado em conteúdo, usando apenas similaridade de gêneros via TF-IDF, com Precision@10 de 0,017172. O valor $\alpha = 1,0$ corresponde a um recomendador puramente colaborativo, usando apenas similaridade no espaço latente do PCA/TruncatedSVD, com Precision@10 de 0,043434.

Comparando esses dois extremos com o melhor ponto encontrado ($\alpha = 0,9$, Precision@10 = 0,044444), observa-se que: (i) o sinal colaborativo isolado já supera em mais de duas vezes o sinal de conteúdo isolado; e (ii) o ganho do modelo híbrido sobre o colaborativo puro é pequeno (+0,00101, cerca de 2,3% de melhora relativa). Isso indica que, embora o conteúdo baseado em gêneros contribua marginalmente, ele está longe de ser a fonte principal de sinal do sistema uma conclusão que cobre, ao menos em parte, a necessidade de um baseline de comparação apontada na Seção 4.6. Ao mesmo tempo, o desempenho cai levemente em $\alpha = 1,0$, sugerindo que a informação de conteúdo ainda acrescenta algum benefício quando usada com peso pequeno.

4.5 Análise qualitativa das recomendações

O notebook inclui testes manuais com filmes escolhidos aleatoriamente e com busca por título parcial. Um dos exemplos finais busca o filme *I Travel Because I Have to, I Come Back Because I Love You* (2009) e solicita cinco recomendações. As recomendações retornadas apresentam `sim_content` = 1,0, pois compartilham o mesmo gênero principal de drama, e `sim_pca` também alto, acima de 0,94. Isso mostra que, para esse exemplo, as duas visões concordam: tanto o conteúdo quanto o espaço latente aproximam filmes de perfil semelhante. Outro exemplo gerado no notebook para *Ace High* (1968) retorna títulos como *Carry On Cowboy* (1966), *Sagebrush Trail* (1933) e *Boot Hill* (1969). Os resultados combinam alta similaridade de conteúdo com proximidade colaborativa, o que é esperado para filmes associados a gêneros e públicos semelhantes.

Um terceiro exemplo, agora com um filme amplamente popular e presente na amostra do heatmap da Figura 2 *Forrest Gump* (1994), revela um padrão distinto dos dois exemplos anteriores. As cinco recomendações retornadas (Tabela 12) apresentam `sim_content` próximo ou igual a 1,0, mas `sim_pca` sensivelmente mais baixo, entre 0,13 e 0,47. Diferentemente dos exemplos de *I Travel Because I Have to, I Come Back Because I Love You* e *Ace High*, nos quais as duas visões concordavam com valores altos, aqui o componente colaborativo contribui pouco para a proximidade encontrada, e o score híbrido é puxado majoritariamente pelo conteúdo. Uma explicação plausível é que filmes extremamente populares, avaliados por um público muito heterogêneo, tendem a ocupar uma posição menos definida no espaço latente: sua vizinhança colaborativa se dilui entre muitos filmes distintos, em vez de se concentrar em poucos títulos muito próximos. Esse resultado reforça a limitação discutida na Seção 3.5 a similaridade por gêneros, por ser um descritor amplo, tende a dominar o score híbrido justamente nos casos em que o sinal colaborativo é mais fraco ou mais disperso.

Tabela 12: Recomendações para *Forrest Gump* (1994)

Filme recomendado	sim_pca	sim_content	sim_hibrida
Train of Life (Train de vie) (1998)	0,226454	1,00000	0,535872
Divine Intervention (Yadon ilaheyya) (2002)	0,190128	1,00000	0,514077
The Virgin Queen (2005)	0,466606	0,58389	0,513519
I Served the King of England (2006)	0,166355	1,00000	0,499813
Tiger and the Snow, The (La tigre e la neve) (2005)	0,130022	1,00000	0,478013

Esses exemplos não provam a qualidade geral do sistema, mas ajudam a verificar se as saídas são plausíveis. A avaliação quantitativa por Precision@K permanece como evidência principal.

4.6 Limitações e informações ausentes

Nenhuma informação indispensável impediu a elaboração do relatório. O notebook contém o pipeline completo, os principais resultados numéricos e as saídas gráficas necessárias. Ainda assim, foram identificadas ausências que limitam a força das conclusões:

- O modelo já contém, implicitamente, dois baselines de comparação ($\alpha = 0$ e $\alpha = 1$), discutidos na Seção 4.4. Um baseline explícito de popularidade pura, isto é, recomendar sempre os filmes mais avaliados independentemente do filme de entrada, ainda não foi implementado e

permanece como extensão futura recomendada.

- A avaliação utiliza Precision@K, mas não inclui métricas complementares como Recall@K, MAP ou NDCG.
- Não há medição formal de tempo de execução, consumo de memória ou custo por etapa.
- O problema de cold start não é tratado; usuários e filmes ausentes do treino são removidos do teste.
- A divisão treino-teste é aleatória por avaliação, não temporal. Ela não reproduz a ordem cronológica real em que um usuário avalia filmes, o que pode superestimar levemente a qualidade percebida do sistema em um cenário de produção, conforme discutido na Seção 4.2.
- Os arquivos PNG citados no README não estavam exportados na raiz do projeto no momento da análise. Para este relatório, as figuras foram recuperadas das saídas incorporadas no próprio notebook `Recomendador de Filmes.ipynb`.
- A busca por título (`buscar_filme`) depende de correspondência de substring contra o campo `title` do MovieLens, que segue a convenção bibliográfica de mover artigos definidos (`The`, `A`, `An`) para o final do título por exemplo, `The Thing (1982)` é armazenado como `Thing, The (1982)`. Isso significa que buscas com o artigo na posição natural da língua falada podem não retornar filmes existentes na base, exigindo que o usuário conheça essa convenção para localizar títulos corretamente. Trata-se de uma limitação de usabilidade da busca, não da qualidade das recomendações geradas.

Apesar dessas limitações, o trabalho apresenta uma implementação coerente e tecnicamente consistente. O pipeline evita densificação, separa treino e teste antes da avaliação final, combina duas fontes de similaridade e testa empiricamente o peso entre elas. A escolha final sugerida pelos experimentos é aumentar o peso do componente colaborativo, com $\alpha = 0,9$, mantendo uma contribuição pequena do conteúdo.

5 Conclusão

Este trabalho investigou como construir um sistema de recomendação de filmes sobre uma base massiva e esparsa o MovieLens ml-32M, combinando duas fontes complementares de sinal: os padrões de coavaliação, capturados via `TruncatedSVD`, e o conteúdo textual dos gêneros, capturado via TF-IDF. As três perguntas colocadas na Motivação encontram respostas concretas no pipeline implementado. A representação da base foi viabilizada por meio de estruturas esparsas (`csr_matrix`) e de busca de vizinhos sob demanda (`NearestNeighbors`), evitando a materialização de qualquer matriz densa de usuários ou de similaridade completa. A comparação entre filmes foi resolvida por um score híbrido, $\text{score_hibrido} = \alpha \cdot \text{sim_pca} + \beta \cdot \text{sim_conteudo}$, que combina as duas visões em uma única medida ponderável. E a qualidade das recomendações foi mensurada de forma quantitativa via Precision@K, sobre uma divisão treino-teste que evita vazamento de dados.

Os experimentos indicam que o componente colaborativo é a principal fonte de sinal do sistema: o extremo puramente colaborativo ($\alpha = 1, 0$) já alcança Precision@10 muito superior ao extremo puramente baseado em conteúdo ($\alpha = 0, 0$), e o modelo híbrido ótimo ($\alpha = 0, 9$) supera o colaborativo puro apenas marginalmente. Isso sugere que, neste dataset, os gêneros funcionam mais como um

refinamento fino do que como uma fonte independente de recomendação um resultado coerente com a natureza ampla e pouco granular dessa forma de representar conteúdo.

Ao mesmo tempo, os resultados absolutos de Precision@K permanecem modestos, o que é esperado dado o grau de esparsidade da base (99,5%) e o protocolo de avaliação adotado, que testa contra interações completamente ocultas. As limitações discutidas na Seção 4.6 ausência de um baseline explícito de popularidade, divisão treino-teste não temporal, tratamento incompleto de cold start e ausência de métricas complementares como Recall@K, MAP ou NDCG não invalidam a arquitetura proposta, mas delimitam com precisão o alcance das conclusões apresentadas.

Como extensões diretas deste trabalho, destacam-se: a adoção de um protocolo de avaliação temporal, mais próximo do uso real do sistema; a inclusão de métricas de ranking complementares à Precision@K; e o enriquecimento da representação de conteúdo com metadados adicionais (sinopse, elenco, diretor), reduzindo a dependência exclusiva dos gêneros. Ainda assim, o sistema desenvolvido demonstra que um recomendador híbrido, tecnicamente simples e computacionalmente viável em ambiente comum, é capaz de organizar um catálogo de mais de 30 mil filmes em vizinhanças interpretáveis e de produzir recomendações consistentes com o comportamento coletivo dos usuários.

Bibliografia

[1] Harper, F. Maxwell; Konstan, Joseph A. **The MovieLens Datasets: History and Context**. ACM Transactions on Interactive Intelligent Systems, 2015. [2] Pedregosa, Fabian et al. **Scikit-learn: Machine Learning in Python**. Journal of Machine Learning Research, 2011. [3] Ricci, Francesco; Rokach, Lior; Shapira, Bracha. **Recommender Systems Handbook**. Springer. [4] Documentação do projeto e notebook `Recomendador de Filmes.ipynb`, analisados localmente em julho de 2026.